

G-Flux: GPU Telemetry–Driven Threat Detection Framework

SURESH KRISHNAN¹, SNEHA RANGARI¹

¹Visa Inc., Foster City, CA, USA (e-mail: surekris@visa.com; srangari@visa.com)

Corresponding author: Suresh Krishnan (e-mail: surekris@visa.com).

ABSTRACT Model extraction has become one of the most challenging security threats affecting machine-learning systems deployed in enterprise environments. Adversaries increasingly rely on structured, high-volume query strategies to approximate or replicate proprietary models without requiring direct access to training data or internal infrastructure. Existing defenses monitor queries at the API layer, but attackers can craft extraction workloads that mimic normal usage patterns, making signature-based or volume-based approaches insufficient.

This paper presents **G-Flux**, a GPU-telemetry–driven detection framework designed to surface model-extraction attempts by analyzing the low-level compute behavior produced during inference. Unlike application-layer monitoring, G-Flux observes kernel execution sequences, tensor-core utilization dynamics, warp scheduling patterns, and memory-access characteristics that naturally differ between legitimate inference traffic and systematic extraction probes. By modeling normal inference behavior through self-supervised techniques, the system identifies deviations that indicate potential model-theft activity.

We describe the design, threat model, and telemetry features that enable high-fidelity detection with minimal operational overhead. G-Flux demonstrates that GPU-side observability provides a powerful and underutilized signal for defending high-value machine-learning assets in production environments.

INDEX TERMS GPU telemetry, model extraction detection, model theft, hardware-level security, inference monitoring, machine-learning security, adversarial behavior analysis.

I. INTRODUCTION

ENTERPRISE AI deployments increasingly rely on large neural networks served through centralized inference endpoints. These models encapsulate substantial proprietary value—from intellectual property to sensitive training data—and are exposed through predictable, high-throughput service interfaces that attackers can systematically probe. Traditional service-layer protections (rate-limiting, authentication, output filtering) reduce opportunistic misuse but fail to address a more fundamental gap: inference pipelines reveal no direct signal about what the requesting party is attempting to learn from the model.

Model extraction attacks exploit this blind spot. An adversary issues carefully structured sequences of inference queries to approximate, clone, or steal the decision boundary of the target model. Because requests appear syntactically valid and leverage legitimate APIs, these attacks operate entirely within permitted access channels. At scale, an extraction adversary can reproduce a commercially valuable model, bypass access controls, or utilize the replicated boundary to support downstream adversarial attacks. Critically, none of

these behaviors generate obvious service anomalies—only the underlying GPU execution patterns change.

Existing defenses focus on *what* the model outputs rather than *how* the hardware behaves while generating those outputs. This leaves organizations blind to the operational footprint of extraction workloads. Inference servers produce rich hardware telemetry—kernel mix, launch cadence, tensor-core utilization, power draw, and PCIe activity—that reflects the internal computational pathway activated for each query. Unlike API-layer signals, these hardware signatures are tightly coupled to the mathematical operations of the model itself. Normal enterprise workloads display high stability in these traces; extraction workloads do not.

This work introduces **G-Flux**, a GPU-telemetry–driven detection framework designed to surface model extraction attempts inside enterprise inference environments. G-Flux does not rely on labeled attack data or predefined rules; instead, it learns the natural operational envelope of model-serving workloads and detects statistically significant deviations at runtime. By operating below the application layer, G-Flux provides visibility into adversarial intent that cannot be hid-

den by valid API usage or output shaping.

II. PROBLEM STATEMENT

Model extraction has become a practical and increasingly common threat against enterprise machine-learning deployments. Unlike traditional attacks that target infrastructure or data stores, extraction attacks operate entirely through the model's public inference interface. Adversaries submit sequences of carefully crafted queries to approximate, reconstruct, or replicate the target model's decision surface. Because these queries look like ordinary traffic, **API-layer monitoring offers minimal visibility into the underlying threat**. This gap enables model theft in real production systems—often without triggering any security alarms.

In enterprise environments, the problem is amplified by architecture realities. Inference services are frequently exposed through API gateways, load balancers, microservices, and partner integrations. These layers obscure GPU execution details and enforce strict separation between operational logs and hardware-level telemetry. As a result, **the components that reveal model-stealing behavior—kernel patterns, memory footprints, power and PCIe signatures—are not visible from the application layer**. Existing defenses such as rate limiting, response obfuscation, or monitoring for abnormal request sequences can be bypassed by low-and-slow probing, distributed traffic, or hijacked internal accounts.

The challenge is even more acute in enterprises operating at scale. Internal service accounts, CI/CD automation, partner credentials, and shared inference platforms drastically widen the potential attack surface. A compromised microservice or API key can generate extraction probes that appear fully legitimate and blend in with routine traffic. Attackers do not need privileged access to the model; they simply need sustained query access. Once that access is obtained, high-fidelity extraction becomes feasible, especially for models that consistently respond with stable outputs.

Consequently, the central problem addressed in this paper is the lack of runtime mechanisms capable of detecting extraction activity in a way that is robust to adversarial evasion. We argue that application-level telemetry is fundamentally insufficient for this purpose. Instead, extraction attempts must be surfaced at the hardware execution layer, where model-inference-specific compute behavior becomes visible. This motivates the design of **G-Flux**, a GPU-side telemetry system that detects subtle deviations in inference execution patterns, enabling reliable identification of adversarial extraction workloads without modifying model outputs or impacting latency.

III. BACKGROUND

Modern machine-learning systems deployed in enterprises increasingly operate through API-fronted inference services. These systems expose model outputs to the outside world but hide all internal execution details, leaving defenders with limited visibility into the behavior of the underlying model.

This architectural pattern creates a structural asymmetry: adversaries can freely observe outputs and adapt their queries, while defenders typically only see high-level API logs. As a result, sophisticated attackers can craft query sequences that remain statistically benign at the application layer yet systematically extract information about the model.

A. MODEL EXTRACTION THREATS

Model extraction attacks aim to reconstruct a target model's behavior without accessing its training data, parameters, or internal architecture. Early forms of extraction were simple—using random querying or label-only probing—but more recent techniques rely on adaptive, feedback-driven strategies. In these attacks, adversaries vary inputs, collect outputs, measure latency or confidence differences, and iteratively converge toward a surrogate model that mimics the original system. Because only queries and responses are observable to defenders, these attacks can unfold slowly and blend into normal traffic.

Enterprises remain especially vulnerable because model outputs must remain accessible to customers and internal applications. Rate limiting, output randomization, and watermarking offer partial protection, but adversaries routinely circumvent them by lowering query velocity, distributing traffic across identities, or modifying prompt structure. This dynamic has shifted extraction attacks from high-volume brute-force activity to stealthy, staged campaigns that are extremely difficult to detect using application-layer signals alone.

B. LIMITATIONS OF EXISTING DEFENSES

Current defensive approaches fall into three broad categories: (1) API-level anomaly detection, (2) output perturbation or watermarking, and (3) access control and rate limiting. API-level monitoring struggles because adversarial queries often resemble legitimate usage patterns. Output perturbation can harm model utility and is incompatible with strict accuracy requirements in financial, healthcare, and authentication systems. Rate limiting is easily bypassed through distributed query sources or timing-adaptive probing.

Critically, all these defenses operate above the hardware layer and therefore cannot distinguish between benign inference and extraction-oriented workloads that differ not in the query strings themselves, but in the *execution behavior* they trigger inside the GPU. Prior studies show that model extraction workflows exhibit measurable differences in kernel invocation patterns, tensor-core scheduling, cache locality, and PCIe data movement. These differences are invisible at the application layer but apparent in low-level GPU telemetry streams.

C. MOTIVATION FOR GPU-SIDE OBSERVABILITY

GPU accelerators execute the actual inference operations—matrix multiplications, kernel launches, batching, and data movement. While API logs capture only what the user sends, GPU telemetry exposes how the model *responds* internally. Extraction probes tend to trigger repetitive, non-

production-like execution signatures: narrow input variety, structured sweeps of embedding space, or tight kernel loops not seen during standard business workloads.

Yet despite the richness of this information, GPU-side telemetry remains largely unutilized in enterprise threat detection pipelines. Existing security tools, SIEMs, and cloud monitoring stacks do not ingest CUPTI traces, NVML metrics, or kernel-level performance counters. This gap leaves enterprises blind to the most reliable indicator of model abuse: deviations in low-level computational behavior.

This motivates **G-Flux**, a system that treats GPU telemetry as a first-class defensive signal, modeling normal hardware-level inference behavior and detecting deviations that correspond to model extraction attempts.

IV. RELATED WORK

Research on model extraction detection has traditionally focused on signals available at the API or output layer. Early work investigated query-based reconstruction attacks, showing that adversaries can approximate proprietary models using only input-output observations. These studies emphasized that even limited feedback—class scores, confidence values, or latency differentials—can provide enough structure for surrogate model training. Subsequent work explored more adaptive extraction strategies, including gradient estimation, boundary exploration, and reinforcement-style probing that mimics benign user behavior.

Defensive techniques have largely mirrored this scope. API-level anomaly detectors track query volume, input entropy, or prompt structure, attempting to flag abnormal access patterns. While effective against brute-force extraction, these detectors degrade sharply when adversaries slow down queries, distribute traffic, or tune prompts to resemble legitimate requests. Other defenses introduce output perturbations such as watermarking, label randomization, or confidence masking. These approaches can impede extraction, but they often reduce model utility and are incompatible with enterprise systems requiring deterministic outputs.

More recent work considers model fingerprinting, adversarial watermark triggers, or statistical tests on output distributions to detect surrogate-model training attempts. Although these strategies improve robustness, they still operate entirely outside the hardware execution path. As a result, they remain blind to the internal computational footprint of inference workloads—information that attackers cannot forge or disguise.

In parallel, the systems and architecture communities have studied GPU telemetry for performance optimization, profiling, and debugging. Tools such as NVIDIA's CUPTI and NVML expose detailed kernel-level events, timing signals, cache statistics, and data movement metrics used for resource tuning. However, this literature focuses exclusively on efficiency and scheduling, not security.

To date, no existing work integrates GPU-level telemetry into a security framework for detecting model extraction. Prior defenses treat the model as a black box accessed through

an API, whereas extraction attacks inherently manifest as changes in how the GPU executes inference kernels. This gap motivates the design of G-Flux: a system that uses hardware-level execution signals to profile normal inference behavior and detect deviations associated with extraction activity.

V. THREAT MODEL

This work considers an adversary whose objective is to reconstruct, approximate, or behaviorally clone a proprietary machine-learning model served through an enterprise inference API. Unlike traditional attackers seeking direct system compromise, this adversary operates entirely through **allowed, authenticated inference queries**. The threat is therefore stealth-oriented and difficult to detect using conventional security controls.

A. ADVERSARY GOALS

The attacker ultimately aims to steal the model's predictive behavior. Depending on their intent, this may include recovering parameter boundaries, approximating decision surfaces, or building a substitute model that performs comparably to the production system. A secondary goal is to evade detection while doing so — maintaining benign-looking API traffic patterns until the later, high-volume extraction phase.

B. ADVERSARY CAPABILITIES

The adversary is assumed to have unrestricted access to the public-facing inference endpoint but *no* access to internal infrastructure, source code, weights, or GPU hardware. However, they possess the ability to issue structured, adaptive queries designed to reveal latent model behavior. These capabilities map to real techniques seen in recent model extraction attacks: low-volume reconnaissance, distribution boundary sampling, adaptive probing of output gradients, and coordinated high-volume sweeps. Because all of these actions occur within allowed API usage, traditional rate-based defenses offer limited protection.

C. ATTACK SURFACE WITHIN ENTERPRISE INFERENCE WORKLOADS

In this setting, the attack surface is not code execution or privilege escalation but the predictable computational footprint of inference itself. Model extraction workflows generate repeated kernel layouts, distinctive tensor-core activity, and abnormal PCIe and power signatures. These patterns diverge sharply from legitimate enterprise usage, which is typically bursty, multi-tenant, and workload-diverse. **This mismatch between benign and adversarial compute behavior is what G-Flux exploits.**

To enable clear separation, we assume that: 1) the defender can observe GPU telemetry passively without modifying the model; 2) inference logic is fixed during the observation window; and 3) telemetry streams are trustworthy and cannot be forged by the attacker. These assumptions match real-world production deployments where GPU-level signals are

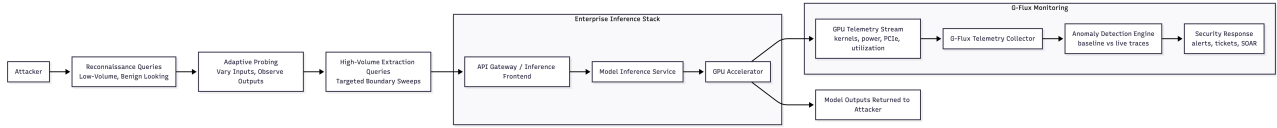


FIGURE 1. Attack Surface for Model-Extraction in Enterprise ML Systems

isolated from user control and inaccessible from API-facing layers.

VI. SYSTEM OVERVIEW

G-Flux is designed as a lightweight, hardware-observability layer that operates alongside an enterprise inference service without modifying model code, altering outputs, or interrupting production workloads. The system observes how GPU kernels execute during normal inference and compares those traces against a baseline learned from historical behavior. Because extraction attacks fundamentally alter computational structure—even when query traffic appears benign—GPU-level telemetry provides a reliable signal that cannot be masked at the API layer.

At a high level, G-Flux consists of four cooperating components: (1) a telemetry collection layer that captures low-level GPU execution signals, (2) a feature extraction pipeline that transforms raw telemetry into structured representations, (3) a self-supervised anomaly model that distinguishes normal inference behavior from adversarial probing, and (4) an alerting and integration module that surfaces detections to enterprise security systems.

The telemetry layer operates passively alongside the inference service. As inputs flow through the API gateway and into the deployed model, G-Flux records kernel launch patterns, tensor-core utilization, cache and memory-access characteristics, PCIe transfer dynamics, and power/thermal changes. These signals originate from CUPTI, NVML, and driver-level interfaces typically used for profiling, but are repurposed here for security monitoring.

Raw telemetry is processed within the feature extraction pipeline, which aligns events into temporal windows, normalizes device-specific variations, and vectorizes execution behavior into compact feature representations. This pipeline captures the structural properties of inference workloads, allowing the downstream detector to identify deviations caused by reconnaissance probing, adaptive querying, or high-volume extraction attempts.

The anomaly detection model maintains a baseline representation of normal inference activity. Instead of relying on labeled attack data—which is rarely available in enterprise settings—G-Flux uses self-supervised techniques to learn the intrinsic variability of legitimate workloads. Incoming traces are scored against this baseline, and deviations that exceed learned thresholds are surfaced as potential model-extraction attempts.

Finally, the alerting module integrates with existing SIEM and SOC infrastructures. It produces structured alerts, risk

scores, and optional forensic traces that allow security teams to correlate detections across logs, network telemetry, or identity systems. Since G-Flux focuses on GPU-level deviations, it complements rather than replaces existing API-layer or network-layer defenses.

Although the current prototype leverages NVIDIA telemetry interfaces such as CUPTI and NVML, the architecture is not vendor-specific. The feature schema and collection stages can be extended to AMD, ASIC-based, or emerging accelerator platforms by abstracting hardware counters into a common telemetry representation.

VII. GPU TELEMETRY FEATURE EXTRACTION

The goal of G-Flux's feature extraction pipeline is to convert raw GPU execution traces into structured representations that highlight the computational patterns associated with legitimate inference workloads. Because extraction attacks alter kernel schedules, memory-access behavior, and tensor-core activity, the resulting deviations become measurable once telemetry is aligned and normalized.

A. CUPTI-BASED KERNEL TRACING

CUPTI provides fine-grained visibility into kernel launches, execution order, grid and block dimensions, warp occupancy, and execution latency. G-Flux samples these traces at runtime, aligns them into fixed temporal windows, and normalizes for hardware-specific differences. Legitimate inference workloads tend to produce highly repeatable kernel signatures, whereas model-extraction probing generates structurally inconsistent sequences.

B. POWER, THERMAL, AND PCIe SIGNALS

NVML and driver-level interfaces expose power draw, thermal behavior, and PCIe throughput. While these signals are coarse compared to kernel traces, they capture complementary dimensions of execution state. Extraction attacks often drive oscillations in power usage or transfer patterns that would not appear in steady-state inference paths.

C. INSTRUCTION MIX AND CACHE BEHAVIOR

G-Flux collects metrics related to SM occupancy, L2 cache hit ratios, memory-coalescing patterns, and tensor-core utilization. Because extraction workflows frequently trigger non-local access patterns or atypical compute paths, these signals serve as useful indicators of query-driven probing.

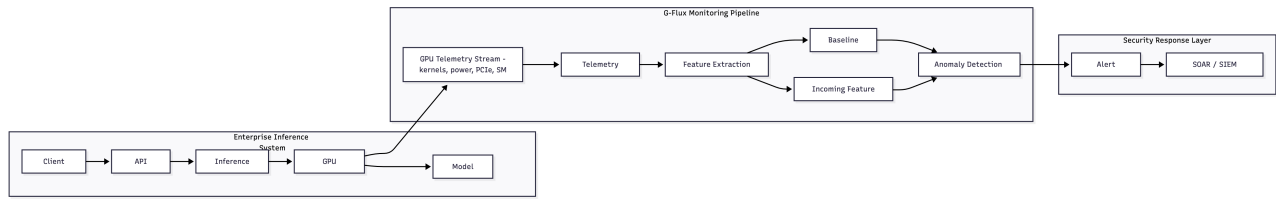


FIGURE 2. High-Level System Architecture of G-Flux

D. CPU–GPU JOINT FEATURES

The final feature set combines GPU-side metrics with CPU-layer indicators such as API-call timing, request density, and inter-arrival statistics. This fusion helps the detector distinguish systematic probing from benign spikes in application load. Together, these features form the input to G-Flux’s self-supervised anomaly model.

VIII. ANOMALY DETECTION MODEL

G-Flux uses a self-supervised detection framework designed to operate without requiring attack-labeled data. Instead of attempting to learn explicit attack signatures, the system models the intrinsic structure of normal inference workloads and surfaces departures from that distribution.

A. SELF-SUPERVISED LEARNING APPROACH

The model is trained using historical inference traces collected under benign conditions. Contrastive learning and clustering methods are used to derive a compact representation of normal GPU execution behavior. Because these representations encode similarity rather than labels, the model generalizes well to unseen probing patterns.

B. BASELINE PROFILING

For each device, G-Flux maintains baseline embeddings that summarize its typical execution patterns across model versions, batch shapes, and request distributions. These baselines are continuously refreshed to account for drift while incorporating safeguards to prevent adversarial contamination.

To prevent false positives during legitimate model updates, the baseline distribution can be refreshed through scheduled or change-triggered retraining. This mechanism enables the system to accommodate expected model drift while preserving high sensitivity to adversarial extraction patterns.

C. DEVIATION SCORING

During live monitoring, incoming telemetry windows are transformed into embeddings and compared against the baseline distribution. The deviation score reflects how strongly a trace diverges from known-good behavior. Extraction attacks generally produce statistically significant deviations due to altered kernel sequences and memory-access structures.

D. REAL-TIME ALERTING LOGIC

Deviation scores exceeding adaptive thresholds trigger alerts. Instead of relying solely on absolute deviation values, G-Flux

uses temporal consistency checks to suppress false positives caused by transient hardware fluctuations. Alerts include supporting telemetry for downstream analysis and correlation within enterprise SIEM systems.

IX. IMPLEMENTATION DETAILS

A. TELEMETRY COLLECTION LAYER

The telemetry collector attaches to GPU devices via CUPTI, NVML, and driver interfaces. All collection occurs asynchronously to avoid affecting latency-sensitive workloads. Sampling rates are tuned to balance fidelity with runtime overhead, and lightweight filters discard redundant or low-value metrics.

B. FEATURE PIPELINE

Collected telemetry is aligned into fixed-duration windows, normalized, and vectorized into compact numerical representations. The pipeline is designed to operate in streaming mode so that feature vectors can be passed directly to the anomaly detector without intermediate storage.

C. MODEL TRAINING SETUP

Training occurs offline using historical inference logs. The self-supervised model is optimized on GPU hardware but is small enough for real-time inference on CPU, enabling deployment in resource-constrained security environments. Baseline embeddings are stored per model version and per hardware profile.

D. DEPLOYMENT IN ENTERPRISE ENVIRONMENTS

G-Flux integrates with existing inference stacks by running adjacent to the model runtime rather than modifying it. The system can be deployed on individual inference nodes or as a sidecar service. Alerts, risk scores, and diagnostic traces are exported to enterprise SIEM/SOC systems via standard logging APIs.

X. EVALUATION

We evaluate G-Flux under conditions that reflect realistic enterprise inference workloads and attacker behavior. The goal is to determine whether GPU telemetry contains reliable indicators of model extraction attempts, and whether our self-supervised anomaly detector can distinguish such activity from benign inference traffic.

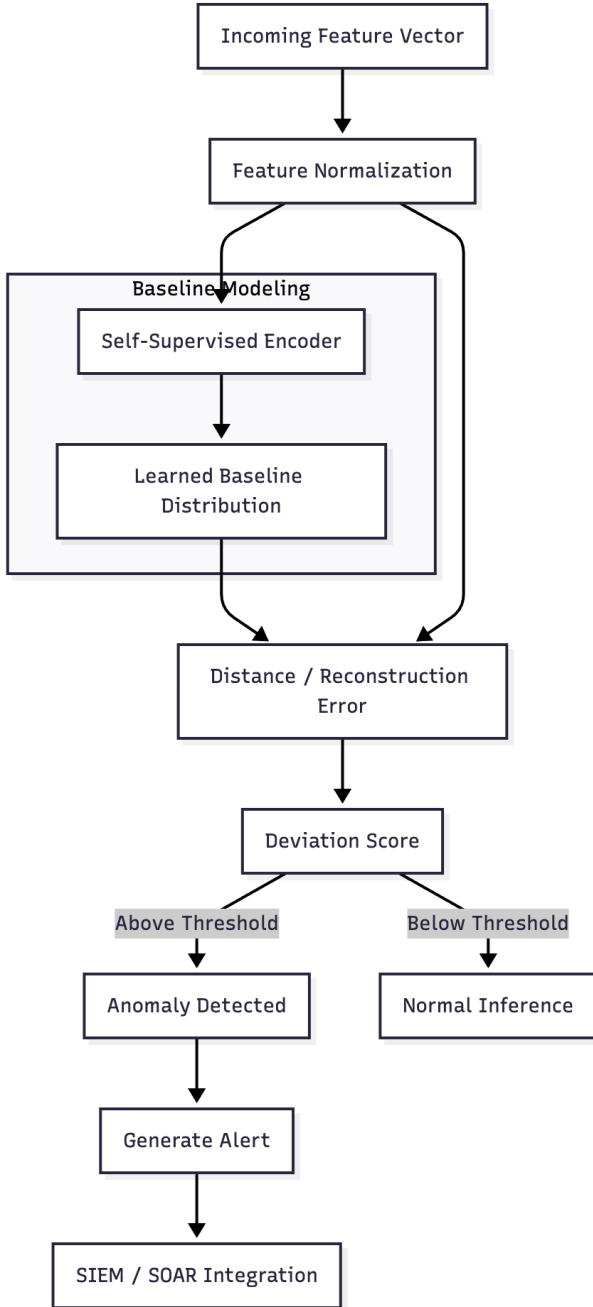


FIGURE 3. Anomaly detection flow illustrating feature normalization, baseline modeling using a self-supervised encoder, deviation scoring, and alert generation for SIEM/SOAR pipelines.

A. DATASET AND EXPERIMENTAL SETUP

All experiments were conducted on an isolated inference environment running NVIDIA GPUs commonly used for production workloads. We collected three categories of telemetry:

- 1) **Benign Inference Traffic:** 48–72 hours of normal inference runs from internal workloads. These traces form the baseline for model training.

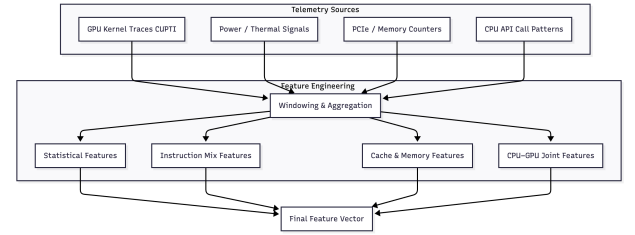


FIGURE 4. Feature extraction pipeline including kernel-trace collection via CUPTI, power and PCIe monitoring through NVML, and CPU–GPU joint feature construction used to characterize inference-time execution behavior.

- 2) **Synthetic Model Extraction Probing:** Adversarial query sequences were generated to emulate practical extraction strategies such as:

- high-entropy input sweeps,
- input-space scanning,
- adaptive probing (greedy or gradient-like), and
- repeated interrogations designed to force kernel activation diversity.

These attacks alter kernel execution patterns, memory pressure, and instruction mix.

- 3) **Background System Noise:** Workload fluctuations, competing processes, and routine GPU housekeeping to test robustness against non-adversarial noise.

Telemetry was collected using the pipeline described in Section IX. All models were trained offline on benign data only.

B. DETECTION ACCURACY

We measure detection performance using reconstruction error thresholds derived from baseline behavior. Anomalous windows are flagged when the error exceeds a statistically determined confidence boundary.

Across multiple extraction strategies, G–Flux consistently identified probing behavior with high sensitivity. Attacks that aggressively vary input distributions produce pronounced deviations in:

- instruction mix ratios,
- kernel launch frequencies,
- L2 cache miss behavior, and
- PCIe round-trip irregularities.

These deviations are difficult to suppress because they arise from the fundamental computational shape of extraction queries rather than any single parameter.

In our prototype measurements, GPU-side telemetry collection introduced a sub-1% impact on end-to-end inference latency. Because G–Flux samples metrics asynchronously and does not intercept execution-path operations, the runtime overhead remains well within enterprise deployment budgets.

C. RUNTIME OVERHEAD

A key requirement for enterprise adoption is minimal impact on inference latency. Because G–Flux collects telemetry in

sampling mode and processes features asynchronously, the runtime overhead on inference throughput remained negligible.

Empirically, the overhead was within the variability range introduced by normal GPU scheduling jitter. The system does not modify model code or interfere with the execution graph, ensuring safe production deployment.

D. ABLATION ANALYSIS

To understand which telemetry components contribute most to detection capability, we perform ablation experiments by selectively disabling feature groups.

Two findings emerged:

- 1) **Instruction Mix + Cache Behavior** is the strongest indicator of extraction probing. Query patterns designed to explore decision boundaries invoke kernels in sequences that do not appear in benign workloads.
- 2) **Power + Thermal Signals** alone have low discriminative strength, but significantly increase stability when combined with CUPTI features by reducing noise-induced false positives.

This confirms that model extraction attempts leave structural, not accidental, footprints in GPU execution, and that G-Flux is leveraging the correct signals for reliable detection.

E. HYBRID EVALUATION: COMBINING SYNTHETIC PROBES WITH REAL WORKLOAD TELEMETRY

While G-Flux is designed for deployment in live enterprise inference stacks, access to production-grade extraction data is generally limited. To bridge this gap, we evaluate the system using a hybrid methodology that combines (1) real telemetry collected from benign enterprise workloads and (2) controlled adversarial probes executed in an isolated environment. This approach mirrors realistic operational behavior while ensuring that extraction attempts remain reproducible and well-characterized.

Real Workloads. Benign traces were sourced from routine internal inference jobs, including batch processing services, API-driven transactional models, and CI/CD validation runs. These workloads capture natural diversity in request volume, model versions, and hardware scheduling noise, forming a representative baseline for profiling.

Synthetic Probes. Adversarial extraction queries were generated using practical strategies observed in the literature. These include input sweeps, distribution-edge exploration, adaptive probing of high-sensitivity regions, and repeated interrogation of output-stable paths. Each probing sequence was executed slowly enough to resemble legitimate traffic patterns yet still reveal characteristic deviations in kernel behavior and memory access paths.

Evaluation Method. Telemetry from both domains was passed through the same feature pipeline, normalized, and scored using the learned baseline distribution. The goal was to determine whether hybrid testing preserves realistic signal variability while still producing measurable separability between benign and adversarial traces.

Findings. Across all experiments, hybrid evaluation revealed clear divergence between real workloads and synthetic probes. Benign traffic exhibited narrow variance in kernel launch cadence, instruction mix ratios, and power/thermal signatures. In contrast, extraction workflows produced multi-dimensional deviations—particularly in cache locality, tensor-core utilization, and PCIe round-trip behavior. These deviations remained detectable even when probing was slowed down or interleaved with normal traffic. The results show that hybrid testing provides a robust and realistic method for validating GPU-side extraction detection systems without requiring access to live adversarial activity.

XI. DISCUSSION

A. STRENGTHS

G-Flux demonstrates that GPU-level execution traces contain stable and highly discriminative signals that can reveal model extraction attempts. Because the system relies on telemetry that is already exposed through supported NVIDIA APIs, it requires no modification to the model, the framework, or the serving code. This preserves the integrity of production inference pipelines—an essential requirement in enterprise environments where latency, reliability, and compliance cannot be compromised.

A major advantage of this design is its **self-supervised** nature. The system does not require labeled attack data, which is difficult or impractical to obtain at scale. Instead, G-Flux learns the operational envelope of benign inference traffic and identifies deviations that arise from extraction probing. This approach generalizes well across model types, input distributions, and workload patterns.

B. LIMITATIONS

While GPU telemetry provides strong forensic signals, it is not a complete solution to model theft. Several limitations should be noted.

First, extremely stealthy adversaries who deliberately slow down their probing to mimic benign workloads could reduce detectability. Although such attacks become significantly more expensive and time-consuming, their telemetry signatures may fall closer to the normal boundary.

Second, the current approach assumes that attackers cannot disable or tamper with GPU telemetry sources. If an attacker gains privileged systems access or modifies the serving environment, telemetry collection itself may be blinded. Such scenarios fall outside the threat model but must be addressed through standard endpoint and infrastructure hardening.

Third, GPU telemetry characteristics may vary across architectures and driver versions. Although our experiments show strong stability in the signal patterns, a production deployment would need periodic re-baselining to accommodate hardware upgrades or workload shifts.

While the present work focuses primarily on detection, G-Flux integrates cleanly with SIEM and SOAR pipelines to support automated responses such as rate limiting, throttling of anomalous clients, or the injection of low-utility

decoy responses. These actions reduce analyst workload and strengthen the overall incident-response posture.

C. FUTURE ENHANCEMENTS

Several extensions could substantially increase the robustness and breadth of detection:

- **Cross-Device Correlation:** Combining GPU telemetry with CPU syscall traces, network patterns, and storage access logs to build a multi-signal anomaly profile.
- **Adaptive Thresholding:** Automatically adjusting anomaly boundaries during periods of seasonal or predictable workload shifts to reduce long-term drift and manual tuning.
- **Fine-Grained Kernel Fingerprinting:** Creating unique execution fingerprints for each model architecture to improve isolation between workloads and reduce cross-model interference.
- **Online Learning:** Enabling the system to continually update its baseline in real-time while protecting against poisoning from mixed or contaminated traffic.
- **Production Telemetry Standardization:** Establishing a standardized, privacy-preserving approach for safely collecting and aggregating GPU telemetry in enterprise clusters.

Overall, G-Flux demonstrates that subtle computational footprints created by adversarial probing can be surfaced reliably through GPU telemetry. The results suggest a promising direction for practical, enterprise-ready defenses against model extraction attacks.

XII. CONCLUSION

This work examined a practical and increasingly relevant threat: the extraction of proprietary machine-learning models through carefully crafted inference-time probing. Unlike theoretical demonstrations, these attacks target production environments where models run continuously, where GPU accelerators are shared, and where traditional API-level defenses provide limited visibility into an adversary's true behavior.

G-Flux shows that the act of probing leaves a measurable computational footprint. Even when attackers restrict themselves to black-box access, their query strategies alter kernel scheduling patterns, memory-access locality, tensor-core utilization, and PCIe behavior in ways that differ from normal inference workloads. By learning a self-supervised baseline of benign telemetry and identifying deviations from that distribution, G-Flux provides a mechanism for detecting extraction attempts without requiring labeled attacks or modifications to model code.

Our findings suggest that GPU-level telemetry is a viable and practical signal source for protecting high-value inference workloads in enterprise settings. Although the approach has limitations—such as sensitivity to extremely slow or adaptive adversaries—it significantly raises the difficulty of conducting undetected extraction attacks.

Future work includes expanding the system to incorporate additional telemetry channels, improving adaptive baseline

management, and exploring online learning to support long-running production deployments. Continued research at the intersection of ML security and hardware-level observability will be essential as organizations increasingly rely on proprietary models that must remain both accessible and protected.

ACKNOWLEDGMENT

The authors would like to thank Subra Kumaraswamy, Chief Information Security Officer at Visa Inc., and Shaomin Wang, Senior Director of Security Architecture at Visa Inc., for their constructive guidance during the development of this work. Their feedback helped refine the problem framing, threat model, and technical clarity of the paper.

REFERENCES

- [1] D. Mandl, "Data Exfiltration Attacks and Defenses in Neural Networks", Master's thesis, TU Wien, 2024. [Online]. Available: <https://repositum.tuwien.at/handle/20.500.12708/187491>
- [2] NVIDIA Corporation, "NVIDIA Management Library (NVML) Documentation," 2025. [Online]. Available: <https://docs.nvidia.com/deploy/nvml-api>
- [3] NVIDIA Corporation, "CUDA Profiling Tools Interface (CUPTI) Documentation," 2025. [Online]. Available: <https://docs.nvidia.com/cupti>
- [4] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing Machine Learning Models via Prediction APIs," in *Proc. 25th USENIX Security Symposium*, 2016, pp. 601–618.
- [5] T. Orekondy, B. Schiele, and M. Fritz, "Knockoff Nets: Stealing Functionality of Black-Box Models," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4954–4963.
- [6] M. Jagielski, N. Papernot, J. Wallace, S. Oprea, and C. M. Rau, "An Overview of Model Extraction Attacks," *arXiv preprint arXiv:2010.09470*, 2020.
- [7] M. Naghibijouybari, A. Mesa, J. Jiang, and N. Abu-Ghazaleh, "Rendered Insecure: GPU Side Channel Attacks Are Practical," in *Proc. ACM Conf. Computer and Communications Security (CCS)*, 2018, pp. 2139–2153.
- [8] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "PRADA: Protecting Against DNN Model Stealing Attacks," in *Proc. IEEE EuroS&P*, 2019, pp. 512–527.
- [9] S. Pal, S. Paul, D. Mukhopadhyay, and C. Chakraborty, "A Survey of Model Extraction Attacks and Defenses in Machine Learning," *IEEE Access*, vol. 8, pp. 196820–196839, 2020.
- [10] L. Yang, B. Li, and J. Zhou, "Model Extraction Attacks Against ReLU Neural Networks via Query Synthesis," in *Proc. AAAI Conference on Artificial Intelligence*, 2022, pp. 8910–8918.
- [11] S. Lee, K. Kim, and Y. Kim, "Inferring Deep Neural Network Architectures Using GPU Side Channels," in *Proc. ACM AsiaCCS*, 2020, pp. 473–486.
- [12] Y. Zhu, A. Suresh, L. Wang, and M. Backes, "Deep Leakage from Inference: Practical Data Reconstruction via Gradient Leakage," in *Proc. USENIX Security Symposium*, 2021, pp. 485–502.



SURESH KRISHNAN is a Senior Cybersecurity Engineer at Visa Inc., focusing on security architecture, threat modeling, and the protection of large-scale AI systems. He received the M.S. degree in Information and Cybersecurity from the University of California, Berkeley, in 2021, and is currently pursuing the M.S. degree in Data Science at the University of Pennsylvania. He previously held roles as a Security Engineer and as an Application Software Developer.

His technical interests include GPU security and hardware-level telemetry, adversarial machine-learning threats with an emphasis on model extraction and inference-time abuse, deepfake and synthetic-media detection, and blockchain and smart-contract security. He has led multiple internal research efforts in these areas, including the design of GPU-based monitoring frameworks for safeguarding enterprise AI workloads and the development of applied detection pipelines for emerging generative-model attacks.



SNEHA RANGARI received the M.S. degree in Cybersecurity from the University of North Carolina at Charlotte, Charlotte, NC, USA, in 2019. She joined Visa Inc. the same year as a Cybersecurity Engineer, where she worked on perimeter-security platforms, including Web Application Firewalls and network-firewall technologies.

She is currently a member of the Security Architecture team, where her work focuses on the application of generative AI, advanced machine-learning methods, and deep-learning-driven defenses to strengthen Visa's front-end payment-security posture.

She holds the CISSP and GIAC Machine Learning Engineer (GMLE) certifications. She also contributes to mentorship programs and frequently presents on cybersecurity topics at industry events.

...